

Data Science Reference Guide

Core concepts, decisions and pitfalls for the Data Scientist badge assessment

TP	FP
FN	TN

CREATED BY

Ahmed Alharbi

a-2011a@hotmail.com

Edition 1 · September 2026

Contents

0	Fundamentals	3	7	Ensembles and Imbalanced Data	24
	0.1 What is data science?	3		7.1 Bagging, boosting, stacking	24
	0.2 Data basics	3		7.2 Random forest	24
	0.3 What is machine learning?	4		7.3 Gradient boosting	24
	0.4 The mathematics you need	4		7.4 Imbalanced classes	24
	0.5 How a model learns: a simple example	5	8	Unsupervised Learning	26
	0.6 The Python toolkit	5		8.1 Clustering algorithms	26
	0.7 A complete mini project	6		8.2 Dimensionality reduction	26
	0.8 Suggested study path	7		8.3 Anomaly detection	26
1	The Data Science Workflow	8	9	Time Series, Text, Recommenders and Deep Learning	27
	1.1 The project lifecycle (CRISP-DM)	8		9.1 Time series	27
	1.2 Framing a business problem as an ML task	8		9.2 Text (NLP)	27
	1.3 Do we need machine learning at all?	9		9.3 Recommender systems	27
2	Statistics Foundations	10		9.4 Neural networks basics	27
	2.1 Describing data	10	10	Evaluating Classification Models	28
	2.2 Common distributions	10		10.1 The confusion matrix	28
	2.3 Sampling and bias	10		10.2 Precision or recall? Decide by the cost of errors	28
	2.4 Central limit theorem and standard error	11		10.3 Thresholds, ROC and PR curves	29
	2.5 Confidence intervals	11		10.4 Calibration	29
	2.6 Correlation and causation	11		10.5 Multi-class metrics	29
	2.7 Probability and Bayes' theorem	12		10.6 Ranking and campaign metrics	30
3	Hypothesis Testing and A/B Testing	13	11	Regression Metrics and Validation Design	31
	3.1 The logic of a test	13		11.1 Regression metrics	31
	3.2 Errors and power	13		11.2 Train, validation and test sets	31
	3.3 Choosing a test	13		11.3 Cross-validation strategies	31
	3.4 A/B testing	13		11.4 Data leakage	32
	3.5 Communicating insights	14		11.5 Comparing models fairly	32
4	Data Exploration and Cleaning	15	12	Production ML (MLOps)	33
	4.1 A practical EDA checklist	15		12.1 Serving patterns	33
	4.2 Missing data	15		12.2 Packaging and reproducibility	33
	4.3 Outliers	16		12.3 Training-serving skew and feature stores	33
	4.4 Transforming skewed variables	16		12.4 Deployment strategies	33
	4.5 Data quality issues	17		12.5 Monitoring and drift	34
5	Feature Engineering and Preprocessing	18		12.6 Scale and cost	34
	5.1 Encoding categorical variables	18	13	Responsible AI	35
	5.2 Feature scaling	18		13.1 Explainability	35
	5.3 Pipelines and preprocessing leakage	18		13.2 Fairness and bias	35
	5.4 Creating useful features	19		13.3 Privacy and data protection	35
	5.5 Multicollinearity	19		13.4 Human oversight and documentation	35
	5.6 Feature selection	19	14	Quick Decision Sheets	36
6	Supervised Learning Models	21		14.1 Which metric?	36
	6.1 Bias-variance trade-off	21		14.2 Which model?	36
	6.2 Linear regression	21		14.3 Which validation?	36
	6.3 Logistic regression	21		14.4 Which preprocessing?	37
	6.4 Regularization	22		14.5 Diagnosing a model	37
	6.5 Decision trees	22	15	Exam Strategy	38
	6.6 Other classic models	22	16	Glossary	39
	6.7 Choosing a model under constraints	23			
	6.8 Hyperparameter tuning	23			

0 Fundamentals

This chapter is for readers starting from zero. It explains the ideas that every later chapter assumes: what data and models are, how a model learns, the minimum mathematics, and the Python tools used in practice. If these feel familiar, skip to Chapter 1.

0.1 What is data science?

Data science turns data into decisions. It combines three skills: **statistics** (reasoning under uncertainty), **programming** (handling data at scale), and **domain knowledge** (knowing which questions matter).

Role	Main question	Typical work
Data analyst	What happened, and why?	SQL, dashboards, reports, descriptive statistics
Data scientist	What will happen, and what should we do?	Experiments, predictive models, insight for decisions
Machine learning engineer	How do we run models reliably in production?	Pipelines, deployment, monitoring, scaling
Data engineer	How does clean data reach everyone on time?	Data pipelines, warehouses, data quality

0.2 Data basics

Most projects start with **tabular data**: a table where each **row** is one observation (a customer, an order, a day) and each **column** is one variable.

Data type	Meaning	Examples
Numerical: continuous	Any value in a range	Price, temperature, delivery time
Numerical: discrete	Countable whole numbers	Number of orders, children, visits
Categorical: nominal	Labels with no order	City, payment method, product type
Categorical: ordinal	Labels with a natural order	Satisfaction (low / medium / high), education level
Date and time	A moment or period	Order timestamp, registration date
Text, images, audio	Unstructured data	Reviews, X-rays, call recordings

- **Feature** (input, independent variable, X): information the model uses to predict.
- **Target** (label, dependent variable, y): the value we want to predict.
- **Grain**: what one row represents. Always confirm it before analysing, because mixing grains (customers and orders in one table) causes wrong totals.
- **Structured data** fits rows and columns. **Unstructured data** (text, images) needs extra processing before most models can use it.

0.3 What is machine learning?

In traditional programming a person writes the rules. In **machine learning** the computer learns the rules from examples: you give it inputs (X) with known answers (y), and it finds a function f so that $f(X) \approx y$ for new data it has never seen.

Type	Data	Goal	Example
Supervised: classification	Labelled, the target is a category	Predict a class	Will this customer churn? (yes / no)
Supervised: regression	Labelled, the target is a number	Predict a quantity	What will this house sell for?
Unsupervised	No labels	Find structure	Group customers into segments
Reinforcement learning	Rewards from actions	Learn a strategy	A robot or game agent learning by trial and error

- **Parameters** are learned from data during training (e.g. the weights of a linear model).
- **Hyperparameters** are chosen by you before training (e.g. tree depth, learning rate).
- **Training** means adjusting the parameters to make the prediction errors, measured by a **loss function**, as small as possible.
- **Generalization** is the real goal: good performance on *new* data, not on the data the model memorized.

THE GOLDEN RULE

Never judge a model on the data it was trained on. Keep a separate **test set** (for example 20% of the rows) aside, and use it only for the final check.

0.4 The mathematics you need

Descriptive statistics. For the values 2, 4, 4, 4, 5, 5, 7, 9:

WORKED EXAMPLE

mean = $40 / 8 = 5$ · squared deviations: 9, 1, 1, 1, 0, 0, 4, 16 → sum = 32

variance (population) = $32 / 8 = 4$ · standard deviation = $\sqrt{4} = 2$

A *sample* variance divides by $n - 1$ instead: $32 / 7 \approx 4.57$, so the standard deviation ≈ 2.14 . The median (middle value) is $(4 + 5) / 2 = 4.5$.

Probability. A probability is a number from 0 (impossible) to 1 (certain).

- **Complement:** $P(\text{not } A) = 1 - P(A)$. If $P(\text{rain}) = 0.3$, then $P(\text{no rain}) = 0.7$.
- **Independent events:** $P(A \text{ and } B) = P(A) \times P(B)$. Two fair coins both land heads with probability $0.5 \times 0.5 = 0.25$.
- **Conditional probability:** $P(A | B) = P(A \text{ and } B) / P(B)$, the probability of A *given* B. If 30 of 100 customers are premium and 12 of those premium customers churned, $P(\text{churn} | \text{premium}) = 12 / 30 = 0.4$.

Vectors and matrices. A **vector** is an ordered list of numbers, such as one customer's features. A **matrix** is a table of numbers. A dataset with n rows and p features is an $n \times p$ matrix.

DOT PRODUCT AND A LINEAR MODEL

$$[2, 3] \cdot [4, 1] = 2 \times 4 + 3 \times 1 = 11$$

$$\text{price} = w \cdot x + b = [3, 20] \cdot [120, 3] + 50 = 360 + 60 + 50 = 470$$

Here x = (size 120 m², 3 rooms), the weights w show how much each feature adds, and b is the intercept. A linear model is just a dot product plus a constant.

Derivatives and gradient descent. A derivative measures the slope: how fast a function changes. Training a model means finding the parameters where the loss is lowest, and **gradient descent** gets there by repeatedly taking small steps downhill.

GRADIENT DESCENT STEP

$$w \leftarrow w - \eta \cdot dL/dw \quad (\eta = \text{learning rate})$$

Example: $L(w) = (w - 3)^2$, so $dL/dw = 2(w - 3)$. Start at $w = 0$ with $\eta = 0.1$: the gradient is -6 , so w becomes 0.6 ; the next gradient is -4.8 , so w becomes 1.08 ; the steps keep shrinking as w approaches the minimum at 3 .

Logarithms and exponentials. $\log$ compresses large numbers ($\log_{10}(1,000) = 3$, $\log_{10}(1,000,000) = 6$), which is why $\log$ transforms tame skewed data such as income. The exponential e^x is its inverse and appears in logistic regression and in growth or decay processes.

0.5 How a model learns: a simple example

Suppose we predict delivery time (minutes) from distance (km) with a straight line: $\text{time} = w \times \text{distance} + b$.

1. **Start** with arbitrary values for w and b .
2. **Predict** the time for every training order.
3. **Measure the error** with a loss function, e.g. mean squared error: the average of $(\text{actual} - \text{predicted})^2$.
4. **Adjust** w and b in the direction that reduces the loss (gradient descent).
5. **Repeat** until the loss stops improving.
6. **Evaluate** the final line on the test set.

Every model in this guide, from logistic regression to neural networks, follows this same loop. What changes is the shape of the function and the loss being minimized.

0.6 The Python toolkit

Library	Used for
NumPy	Fast arrays and mathematical operations
pandas	Loading, cleaning, joining and summarizing tables (DataFrames)
Matplotlib / seaborn	Charts for exploration and reporting

Library	Used for
scikit-learn	Preprocessing, classic models, evaluation, pipelines
XGBoost / LightGBM	Gradient boosting for tabular data
PyTorch / TensorFlow	Deep learning

Everyday pandas operations

```
import pandas as pd
df = pd.read_csv("customers.csv")      # load a table
df.head()                             # first rows
df.info()                             # column types and non-null counts
df.describe()                         # summary statistics for numeric columns
df.isna().sum()                       # missing values per column
df["city"].value_counts()             # frequency of each category
df.groupby("city")["spend"].mean()    # average spend per city
df = df.merge(orders, on="customer_id", how="left") # join two tables
```

The scikit-learn pattern. Almost every object follows the same interface:

- `fit(X, y)` learns from training data.
- `predict(X)` or `predict_proba(X)` makes predictions.
- `transform(X)` applies a learned preprocessing step (scaling, encoding).

0.7 A complete mini project

Goal: predict which telecom customers will churn, so the retention team can call them first.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, roc_auc_score

df = pd.read_csv("churn.csv")
X, y = df.drop(columns=["customer_id", "churn"], df["churn"]

# 1. hold out a test set, keeping the churn rate equal in both parts
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42)

# 2. preprocessing for each column type
num = ["tenure_months", "monthly_charge", "support_calls"]
cat = ["contract_type", "payment_method"]
prep = ColumnTransformer([
    ("num", Pipeline([("impute", SimpleImputer(strategy="median")),
                     ("scale", StandardScaler())]), num),
    ("cat", OneHotEncoder(handle_unknown="ignore"), cat)])
```

```
# 3. one pipeline = preprocessing + model, fitted on training data only
model = Pipeline([("prep", prep),
                  ("clf", LogisticRegression(class_weight="balanced", max_iter=1000))])
model.fit(X_train, y_train)

# 4. evaluate on data the model has never seen
proba = model.predict_proba(X_test)[: , 1]
print(classification_report(y_test, model.predict(X_test)))
print("ROC-AUC:", round(roc_auc_score(y_test, proba), 3))
```

Step	Why it matters	Read more
Define the goal and the label	"Churn" must be measurable and the prediction must lead to an action	Chapter 1
Stratified train/test split	Honest evaluation with the same class balance in both parts	Chapter 11
Imputation, scaling, encoding in a Pipeline	Prepares the data without leaking test information	Chapters 4–5
A simple, interpretable baseline model	Sets the bar that more complex models must beat	Chapter 6
<code>class_weight="balanced"</code>	Churners are the minority class	Chapter 7
Precision, recall, ROC-AUC	Accuracy is misleading when classes are imbalanced	Chapter 10

Next steps in a real project: try a stronger model (random forest, gradient boosting) with cross-validation, choose a decision threshold from the cost of calls vs lost customers, explain the drivers of churn to the business, then deploy and monitor the model.

0.8 Suggested study path

If you are...	Read in this order
A complete beginner	Chapter 0 → 2 → 4 → 5 → 6 → 10 → 11, then the rest
Comfortable with Python, new to ML	Chapters 1 → 4 → 5 → 6 → 7 → 10 → 11 → 12
Revising just before the exam	Chapter 14 (decision sheets), 15 (strategy), then the "Common trap" boxes in every chapter

LEARN BY DOING

Reading is not enough. Open a Jupyter notebook, load a public dataset (Kaggle's Titanic or a telecom churn dataset), and reproduce the mini project above. Then change one thing at a time and watch how the metrics move.

1 The Data Science Workflow

Most project failures happen before any model is trained: the wrong question, the wrong label, or a success measure nobody agreed on. Good data scientists spend real effort on framing.

1.1 The project lifecycle (CRISP-DM)

CRISP-DM is the most widely used process model. It is iterative: later stages often send you back to earlier ones.

Stage	Key question	Typical outputs
Business understanding	What decision will this support, and how will we measure success?	Objective, KPI, constraints, success threshold
Data understanding	What data exists, how good is it, and is the target available?	Data inventory, quality report, first EDA
Data preparation	How do we turn raw data into a modelling table?	Cleaned features, labels, train/validation/test splits
Modelling	Which approach solves the task under the constraints?	Baseline, candidate models, tuned model
Evaluation	Does it meet the business goal, not just the metric?	Test results, error analysis, go/no-go decision
Deployment	How is it used, monitored and maintained?	Service or batch job, monitoring, retraining plan

EXAM TIP

When a question asks what to do *first*, the answer is usually about understanding the problem or the data (clarify the objective, check label availability, inspect data quality), not about choosing an algorithm.

1.2 Framing a business problem as an ML task

Business request	ML formulation	Typical target / output
Which customers will leave next month?	Binary classification	churn = 1 / 0 within a defined horizon
How many units will we sell per store per week?	Regression / time-series forecasting	Numeric quantity
Group customers with similar behaviour	Clustering (unsupervised)	Segment ID
Flag unusual transactions with few labelled cases	Anomaly detection (or classification if labels exist)	Anomaly score
Which products should we show each user first?	Ranking / recommendation	Ordered list

Business request	ML formulation	Typical target / output
Which of five ticket categories is this complaint?	Multi-class classification	One of K classes

- **Define the label precisely:** "churned" must mean something measurable (e.g. no activity for 30 days) with a clear prediction horizon.
- **Only use information available at prediction time.** A feature created after the outcome causes *target leakage*.
- **Link the model metric to a business KPI.** Recall of fraud cases matters because each missed fraud costs money.
- **Start with a baseline:** a simple rule, the majority class, or last period's value. A model must beat it to be worth deploying.

1.3 Do we need machine learning at all?

Machine learning is justified when the pattern is too complex for explicit rules, enough relevant historical data exists, the pattern is reasonably stable, and errors are tolerable or can be reviewed. Prefer rules, SQL or a dashboard when the logic is simple and known, when there is little data, or when a fully transparent deterministic decision is required.

COMMON TRAP

Choosing a complex model before confirming that a simple rule or a baseline already solves the problem. The best answer often scopes a small MVP first.

2 Statistics Foundations

Statistics lets you describe data honestly, quantify uncertainty, and avoid being fooled by noise.

2.1 Describing data

Measure	What it describes	When to prefer it
Mean	Arithmetic average	Symmetric data without extreme values
Median	Middle value (50th percentile)	Skewed data or outliers (income, house prices, delivery times)
Mode	Most frequent value	Categorical data
Standard deviation	Typical distance from the mean	Roughly symmetric data
IQR (Q3 – Q1)	Spread of the middle 50%	Skewed data; robust to outliers
Percentiles	Value below which p% of data falls	Service levels (e.g. 95th percentile latency)

- **Right (positive) skew:** long tail to the right, and usually mean > median (income, transaction amounts).
- **Left (negative) skew:** long tail to the left, and usually mean < median.
- **Bimodal:** two peaks, often a sign of two mixed sub-populations that should be analysed separately.

2.2 Common distributions

Distribution	Models	Example
Normal	Sums of many small effects; symmetric bell	Measurement error, heights
Binomial	Number of successes in n independent yes/no trials	Conversions out of 1,000 visitors
Poisson	Count of events in a fixed interval at a constant rate	Calls per hour, defects per batch
Exponential	Waiting time between Poisson events	Time until the next arrival
Long-tailed / log-normal	Many small values, few very large ones	Revenue per customer, file sizes

NORMAL DISTRIBUTION RULE OF THUMB

≈68% within $\pm 1\sigma$ · ≈95% within $\pm 2\sigma$ · ≈99.7% within $\pm 3\sigma$

2.3 Sampling and bias

- **Simple random sampling:** every unit has an equal chance of selection.
- **Stratified sampling:** sample within groups (strata) to guarantee each group is represented. It is essential for rare groups.
- **Cluster sampling:** randomly select whole groups (e.g. branches), then observe everyone inside them. It is cheaper but less precise.

- **Convenience sampling:** whoever is easiest to reach. It is prone to bias.

Bias	What goes wrong
Selection bias	The sample differs systematically from the population (e.g. surveying only app users)
Survivorship bias	Only "survivors" are observed (e.g. analysing only companies still in business)
Non-response bias	People who respond differ from people who do not
Measurement bias	The measuring instrument or process distorts values

2.4 Central limit theorem and standard error

The **central limit theorem** says the distribution of the sample mean is approximately normal for large enough samples, whatever the shape of the original data. This is why confidence intervals and many tests work.

STANDARD ERROR OF THE MEAN

$$SE = \sigma / \sqrt{n}$$

Quadrupling the sample size halves the standard error.

2.5 Confidence intervals

A 95% confidence interval is produced by a procedure that captures the true parameter in 95% of repeated samples.

APPROXIMATE 95% CI FOR A MEAN

$$\bar{x} \pm 1.96 \times SE$$

- A larger sample gives a narrower interval, and a higher confidence level gives a wider one.
- If a 95% CI for a difference excludes 0, the difference is significant at $\alpha = 0.05$.

COMMON TRAP

"There is a 95% probability that the true value lies in this specific interval" is not the frequentist interpretation. The 95% describes the long-run behaviour of the procedure.

2.6 Correlation and causation

- **Pearson correlation** measures *linear* association and is sensitive to outliers.
- **Spearman correlation** measures *monotonic* association using ranks, so it is robust to outliers and to non-linear but monotonic relationships.
- A correlation near 0 does not mean "no relationship". A strong U-shaped pattern can have a correlation near 0.
- **Correlation is not causation.** A *confounder* can drive both variables (e.g. hot weather raises both ice-cream sales and drowning incidents).

- To establish causation, run a randomized controlled experiment when possible.

SIMPSON'S PARADOX

A trend that appears in every subgroup can reverse when the groups are combined, because group sizes differ.

Example: a hospital has higher survival in both mild and severe cases but lower overall survival, because it treats far more severe cases. Always check whether a lurking variable changes the composition of the groups.

Regression to the mean: extreme observations tend to be followed by values closer to the average. Branches selected for "worst performance" often improve next quarter even without any intervention.

2.7 Probability and Bayes' theorem

BAYES' THEOREM

$$P(A \mid B) = P(B \mid A) \cdot P(A) / P(B)$$

Base-rate example. A disease has 1% prevalence. The test has 90% sensitivity and 91% specificity. Among 10,000 people:

- 100 are sick → 90 test positive (true positives).
- 9,900 are healthy → 9% of them, 891, test positive (false positives).
- $P(\text{sick} \mid \text{positive}) = 90 / (90 + 891) \approx \mathbf{9.2\%}$.

COMMON TRAP

Confusing sensitivity, $P(\text{positive} \mid \text{sick})$, with the positive predictive value, $P(\text{sick} \mid \text{positive})$. When the condition is rare, most positives can be false.

Expected value = $\Sigma (\text{outcome} \times \text{probability})$. It is used to compare decisions under uncertainty, such as the expected profit of contacting a customer.

3 Hypothesis Testing and A/B Testing

3.1 The logic of a test

- **Null hypothesis (H_0):** no effect or no difference.
- **Alternative hypothesis (H_1):** an effect exists.
- **p-value:** the probability of seeing data at least this extreme *if H_0 were true*.
- **Significance level α** (often 0.05): reject H_0 if $p < \alpha$.

WHAT A P-VALUE IS NOT

- It is not the probability that H_0 is true.
- It is not the size or importance of the effect.
- $p > 0.05$ does not prove there is no effect. The test may lack power.

3.2 Errors and power

	H_0 true	H_0 false
Reject H_0	Type I error (false positive), probability α	Correct decision (power = $1 - \beta$)
Do not reject	Correct decision	Type II error (false negative), probability β

Power increases with a larger sample, a larger true effect, lower variance, and a higher α . Aim for 80% power when planning an experiment.

3.3 Choosing a test

Situation	Test
Compare the means of two independent groups	Two-sample t-test (Welch's if variances differ)
Compare two measurements on the same units (before/after)	Paired t-test
Compare the means of 3 or more groups	ANOVA, then post-hoc tests
Two groups, skewed data or ordinal scale	Mann–Whitney U (non-parametric)
Association between two categorical variables	Chi-square test of independence
Compare two proportions (conversion rates)	Two-proportion z-test (or chi-square)

3.4 A/B testing

1. **Define one primary metric** and guardrail metrics (e.g. latency, refunds) *before* starting.
2. **Choose the randomization unit** (user, session, store). Randomizing by user avoids one person seeing both versions.

3. **Compute the sample size** from the baseline rate, the minimum detectable effect, α and power.
4. **Run for the planned duration**, at least full weekly cycles, to cover day-of-week effects.
5. **Analyse once** at the end, and report the effect size with a confidence interval.

Pitfall	Why it matters	Remedy
Peeking and stopping early	Inflates false positives	Fixed horizon, or sequential testing methods
Many metrics or segments	At $\alpha = 0.05$, 1 in 20 tests is "significant" by chance	Pre-register a primary metric; correct with Bonferroni (α/m) or FDR
Novelty effect	Users react to change itself, and the effect fades	Run longer; look at the effect over time
Sample ratio mismatch	The observed split (e.g. 52/48) differs from the plan (50/50)	Investigate the assignment bug before trusting results
Network or interference effects	Treated users affect control users	Randomize by cluster (city, group)

EXAM TIP

Statistical significance is not the same as practical significance. With millions of users a tiny, commercially useless lift can be "significant". Compare the effect size with the cost of implementing the change.

3.5 Communicating insights

- Lead with the decision or recommendation, then the evidence.
- Report uncertainty (ranges, confidence intervals), not only point estimates.
- Choose the right chart: a histogram or box plot for a distribution, a line chart for a trend over time, a bar chart for comparing categories, a scatter plot for the relationship between two numeric variables, and a heatmap for a correlation matrix.
- Avoid misleading visuals: truncated axes on bar charts, dual axes, 3-D effects, and inconsistent scales.

4 Data Exploration and Cleaning

Exploratory data analysis (EDA) tells you what the data can and cannot support. Cleaning decisions should be justified, documented and reproducible.

4.1 A practical EDA checklist

1. **Shape and grain:** how many rows, and what does one row represent (a customer, a transaction, a day)?
2. **Types:** are numbers stored as text? Are dates parsed? Are IDs mistakenly treated as numeric?
3. **Missing values:** the share per column, and whether missingness is related to other variables or the target.
4. **Distributions:** skew, impossible values (negative age), sentinel codes (–999, 9999).
5. **Duplicates:** exact duplicates and duplicates on the business key.
6. **Target:** class balance, or the distribution of the numeric target.
7. **Relationships:** correlations, feature vs target plots, and groups that behave differently.
8. **Time:** coverage, gaps, and changes in definitions over time.

COMMON TRAP

Joins that silently multiply rows. If a join key is not unique in the right-hand table, a LEFT JOIN duplicates rows and inflates totals. Check key uniqueness before joining.

4.2 Missing data

Mechanism	Meaning	Example	Implication
MCAR: missing completely at random	Missingness is unrelated to any variable	A random sensor transmission failure	Deletion is unbiased (you only lose data)
MAR: missing at random	Missingness depends on <i>observed</i> variables	Older customers skip the "app rating" field	Model-based imputation using those variables works well
MNAR: missing not at random	Missingness depends on the <i>missing value itself</i>	High earners refuse to report income	Hard to fix; add a missing indicator, collect data, or use domain knowledge

Strategy	Use when	Risk
Drop rows (listwise deletion)	Few rows affected and data is MCAR	Losing data; bias if not MCAR
Drop the column	Mostly missing (e.g. >80%) and of little value	Losing a useful signal if missingness is informative
Mean or median imputation	Simple baseline; median for skewed data	Shrinks variance and distorts relationships
Mode or "Unknown" category	Categorical features	May create a large artificial category
KNN or iterative (model-based) imputation	MAR data with related features	Slower; must be fitted on training data only

Strategy	Use when	Risk
Missing-indicator feature	Missingness itself carries information	Adds features
Forward fill or interpolation	Ordered time series	Wrong across long gaps

EXAM TIP

Fit every imputer on the **training set only** (inside a Pipeline), then apply it to the validation and test sets. Otherwise you leak test information.

4.3 Outliers

IQR RULE

$IQR = Q3 - Q1$ · lower fence = $Q1 - 1.5 \cdot IQR$ · upper fence = $Q3 + 1.5 \cdot IQR$

Example: $Q1 = 20$ and $Q3 = 40$ give $IQR = 20$ and fences of -10 and 70 , so a value of 75 is flagged.

Z-SCORE

$z = (x - \mu) / \sigma$ (often flagged when $|z| > 3$)

The mean and standard deviation are themselves pulled by outliers, so the z-score can miss them in small or skewed samples. The IQR or a robust (median/MAD) z-score is safer.

First decide whether an outlier is an error or a genuine rare event.

Situation	Treatment
Data-entry or sensor error (age 250)	Correct it, or set it to missing and impute
Genuine but extreme values that distort the model	Cap/winsorize, log-transform, or use robust models or losses
Rare events are the target (fraud)	Keep them. They are the signal.

- **Sensitive to outliers:** linear regression (OLS), k-means, kNN, PCA, MSE/RMSE, StandardScaler.
- **Robust:** tree-based models (splits depend on order, not magnitude), the median, MAE, RobustScaler, Huber loss.

4.4 Transforming skewed variables

- **Log transform** ($\log(x + 1)$ when there are zeros) compresses long right tails such as income or prices.
- **Box-Cox** requires strictly positive values. **Yeo-Johnson** also handles zero and negative values.
- Tree-based models rarely need these transforms. Linear and distance-based models often benefit.

4.5 Data quality issues

Issue	Example	Fix
Inconsistent categories	"Riyadh", "riyadh ", "RUH"	Normalize case and whitespace; map to a canonical value
Mixed units	Weights in kg and g	Convert to a single unit
Sentinel values	–999 meaning "unknown"	Replace with missing before computing statistics
Wrong types	"1,250" stored as text	Parse and cast; validate ranges
Mixed date formats	03/04/2025 read as March or April	Parse with an explicit format; use ISO 8601
Duplicate records	The same order ingested twice	Deduplicate on the business key

5 Feature Engineering and Preprocessing

5.1 Encoding categorical variables

Encoder	How it works	Best for	Watch out
One-hot	One binary column per category	Nominal categories with few levels; linear models	High cardinality → many sparse columns
Ordinal / label	Integer code per category	Truly ordered categories (low < medium < high); tree models	Imposes a false order on nominal data for linear and distance models
Target (mean) encoding	Replaces a category with the mean target value	High-cardinality features (thousands of merchants)	Leakage unless done out-of-fold with smoothing
Frequency / count	Replaces a category with its frequency	High cardinality; tree models	Different categories can share a count
Hashing	Hashes categories into a fixed number of columns	Very high cardinality and streaming data	Collisions; not interpretable

- **Dummy-variable trap:** with an intercept in a linear model, drop one one-hot column (`drop_first`) to avoid perfect multicollinearity.
- **Unseen categories at inference:** use `handle_unknown="ignore"` or an "Other" bucket.
- **Rare categories:** group levels below a frequency threshold into "Other".

5.2 Feature scaling

Scaler	Formula	Use when
StandardScaler	$(x - \text{mean}) / \text{std}$	The default for linear models, SVM, PCA, neural networks
MinMaxScaler	$(x - \min) / (\max - \min) \rightarrow [0, 1]$	A bounded range is needed (e.g. neural network inputs, image pixels); no strong outliers
RobustScaler	$(x - \text{median}) / \text{IQR}$	Features with outliers
MaxAbsScaler	$x / \max x $	Sparse data (keeps zeros)

- **Needs scaling:** kNN, k-means, SVM, PCA, regularized linear or logistic regression, neural networks, and anything using gradient descent or distances.
- **Does not need scaling:** decision trees, random forests, gradient boosting.

5.3 Pipelines and preprocessing leakage

Rule: split first, then fit every transformation (imputer, scaler, encoder, feature selector, resampler) on the training data only.

```
pipe = Pipeline([
    ("prep", ColumnTransformer([
```

```

("num", Pipeline([("impute", SimpleImputer(strategy="median")),
                  ("scale", StandardScaler())]), numeric_cols),
("cat", OneHotEncoder(handle_unknown="ignore"), categorical_cols))),
("model", LogisticRegression(max_iter=1000))]
cross_val_score(pipe, X_train, y_train, cv=5) # preprocessing refit inside every fold

```

COMMON TRAP

Scaling or selecting features on the full dataset before cross-validation. The validation folds have then influenced the preprocessing, so the scores are optimistic.

5.4 Creating useful features

- **Datetime:** hour, day of week, month, weekend or holiday flags, time since the last event, and seasonal periods such as Ramadan.
- **Cyclical encoding:** represent the hour as $\sin(2\pi \cdot h/24)$ and $\cos(2\pi \cdot h/24)$, so that 23:00 and 00:00 are close.
- **Aggregations** from event tables: counts, sums, averages, recency and frequency per customer. Use only events before the prediction cutoff.
- **Ratios and interactions:** debt-to-income, price per square metre, spend per visit.
- **Binning:** converts a numeric feature into ranges. It can help interpretability but loses information.
- **Text:** bag-of-words or TF-IDF with n-grams; clean case and punctuation; remove stop words when appropriate.

TF-IDF

$$\text{tfidf}(t, d) = \text{tf}(t, d) \times \log(N / \text{df}(t))$$

A term scores high when it is frequent in a document but rare across the corpus.

5.5 Multicollinearity

Highly correlated predictors make linear-model coefficients unstable and hard to interpret, even when predictions stay accurate.

VARIANCE INFLATION FACTOR

$$\text{VIF}_j = 1 / (1 - R_j^2)$$

R_j^2 comes from regressing feature j on the other features. A VIF above 5–10 signals a problem, so drop or combine features, or use ridge regression.

5.6 Feature selection

Family	Examples	Notes
Filter	Variance threshold, correlation, chi-square, mutual information	Fast and model-agnostic; ignores feature interactions

Family	Examples	Notes
Wrapper	Recursive feature elimination (RFE), forward/backward selection	Uses a model; expensive; risk of overfitting
Embedded	L1 (Lasso) coefficients, tree importances	Selection happens during training
Permutation importance	Drop in validation score when a feature is shuffled	Model-agnostic and reliable; correlated features share importance

Curse of dimensionality: as the number of features grows, data becomes sparse and distances become less meaningful. Distance-based methods suffer most. Remedies are feature selection, dimensionality reduction, and regularization.

6 Supervised Learning Models

6.1 Bias–variance trade-off

	Underfitting (high bias)	Overfitting (high variance)
Training error	High	Low
Validation error	High, close to the training error	Much higher than the training error
Learning curve	Both curves plateau at a high error; more data will not help	A large gap that narrows as data grows; more data helps
Remedies	More complex model, more or better features, less regularization, train longer	More data, regularization, simpler model, feature selection, early stopping, dropout, pruning

READING LEARNING CURVES

If the validation error is still falling as training size grows and the gap is large, collect more data. If both curves are flat and high, change the model or features instead.

6.2 Linear regression

This model predicts a number as a weighted sum of features, fitted by minimizing squared error (OLS).

- **Assumptions:** linear relationship, independent errors, constant error variance (homoscedasticity), and roughly normal residuals (needed for inference). Also avoid strong multicollinearity.
- **Diagnostics:** a residuals-vs-fitted plot. A funnel shape means heteroscedasticity, so try a log target. A curve means a missing non-linearity, so add terms.
- **Coefficient:** the expected change in y for a one-unit increase in x , holding the other features constant.

6.3 Logistic regression

MODEL

$$p = 1 / (1 + e^{-z}), \quad z = \beta_0 + \beta_1 x_1 + \dots \quad \cdot \quad \log\text{-odds} = z$$

A coefficient β multiplies the odds by e^β for each one-unit increase in the feature. When $z = 0$, $p = 0.5$.

- It is a strong baseline for classification: fast, well-calibrated probabilities, and interpretable, which suits regulated decisions such as credit.
- Its decision boundary is linear. Non-linear patterns require engineered features.
- Trained with log loss (cross-entropy).

6.4 Regularization

Type	Penalty	Effect	Use when
L1 (Lasso)	$\lambda \cdot \sum \beta $	Pushes some coefficients exactly to 0 → feature selection	Many features, few truly useful
L2 (Ridge)	$\lambda \cdot \sum \beta^2$	Shrinks all coefficients smoothly; handles correlated features	Multicollinearity; many small effects
ElasticNet	Mix of L1 and L2	Sparse and stable with correlated groups	Many correlated features

- A larger λ (alpha) means stronger regularization, which gives a simpler model with more bias and less variance.
- In scikit-learn's LogisticRegression and SVM, $C = 1/\lambda$, so a *smaller* C means *stronger* regularization.
- Scale features before regularizing so the penalty treats them equally.

6.5 Decision trees

- A tree splits data with if/then rules chosen to maximize purity. It uses Gini impurity or entropy for classification and variance (MSE) reduction for regression.
- **Strengths:** interpretable, handles non-linearity and interactions, needs no scaling, and is robust to outliers in features.
- **Weaknesses:** deep trees overfit, and small data changes can produce a very different tree (instability).
- **Control complexity** with max_depth, min_samples_leaf, min_samples_split, and cost-complexity pruning (ccp_alpha).

GINI IMPURITY

$$\text{Gini} = 1 - \sum p_k^2$$

A node with 60% class A and 40% class B has $\text{Gini} = 1 - (0.36 + 0.16) = 0.48$. A pure node has $\text{Gini} = 0$.

6.6 Other classic models

Model	Key idea	Strengths	Weaknesses
k-nearest neighbours	Predict from the k closest training points	Simple, no training phase, non-linear	Slow prediction on large data; needs scaling; suffers in high dimensions. Small k overfits, large k underfits.
Support vector machine	Maximum-margin boundary; kernels make it non-linear	Effective in high dimensions and on medium-sized data	Scales poorly to very large datasets; needs scaling and tuning of C and gamma
Naive Bayes	Bayes' theorem assuming independent features	Very fast; strong baseline for text	The independence assumption makes probabilities poorly calibrated

SVM parameters: a large C means a narrow margin and fewer training errors (risk of overfitting). A large gamma (RBF kernel) makes each point's influence local, giving a wiggly boundary (risk of overfitting).

6.7 Choosing a model under constraints

Constraint	Good choices
Must explain each decision to regulators or customers	Logistic or linear regression, shallow trees, scorecards
Tabular data, best accuracy	Gradient boosting (XGBoost, LightGBM, CatBoost), random forest
Very low latency or edge device	Linear models, small trees, compressed models
Small dataset	Simple, regularized models; strong cross-validation
Images, audio, free text at scale	Deep learning, usually transfer learning from pretrained models

6.8 Hyperparameter tuning

- **Grid search:** tries every combination. Cost = (product of values per parameter) × (number of folds). It explodes quickly: $4 \times 3 \times 5$ values with 5-fold CV = 300 fits.
- **Random search:** samples combinations and usually finds good settings faster when only a few parameters matter.
- **Bayesian optimization** (e.g. Optuna): uses past results to choose promising settings, which makes it efficient for expensive models.
- **Successive halving / Hyperband:** gives many candidates a small budget, then keeps only the best.
- Tune on validation data or with cross-validation. Never tune on the test set.

MULTI-CLASS STRATEGIES

One-vs-rest trains K binary models. One-vs-one trains $K(K-1)/2$ models, so 45 for 10 classes. Softmax (multinomial) models all K classes jointly.

7 Ensembles and Imbalanced Data

7.1 Bagging, boosting, stacking

	Bagging (e.g. random forest)	Boosting (e.g. XGBoost, LightGBM)	Stacking
Training	Independent models trained in parallel on bootstrap samples	Sequential models; each corrects the errors of the previous ones	A meta-model learns to combine the predictions of base models
Mainly reduces	Variance	Bias (and variance)	Varies
Overfitting risk	Low; more trees do not overfit	Higher; too many rounds overfit without early stopping	Needs out-of-fold predictions
Tuning effort	Low	Higher	High

7.2 Random forest

- Each tree sees a bootstrap sample and a random subset of features at each split (`max_features`). This **de-correlates the trees**, so averaging them reduces variance.
- **Out-of-bag (OOB) score**: each tree is evaluated on the roughly 37% of rows it did not see, giving a free validation estimate.
- Impurity-based feature importance is biased toward high-cardinality and continuous features. Prefer permutation importance or SHAP.

7.3 Gradient boosting

- **Learning rate vs number of trees**: a smaller learning rate needs more trees but usually generalizes better.
- **Early stopping**: stop adding trees when the validation score stops improving.
- **Controls overfitting**: `max_depth`, `min_child_weight`, `subsample (rows)`, `colsample_bytree (features)`, and L1/L2 regularization.
- XGBoost and LightGBM handle missing values natively. CatBoost handles categorical features natively. LightGBM is very fast on large data.
- **Overfitting sign**: the training loss keeps falling while the validation loss starts rising.

7.4 Imbalanced classes

Technique	How	Notes
Use the right metric	Precision, recall, F1, PR-AUC instead of accuracy	Always the first step
Class weights	Penalize minority-class errors more (<code>class_weight="balanced"</code> , <code>scale_pos_weight</code>)	No data change; simple and effective

Technique	How	Notes
Threshold moving	Choose the probability cut-off from business costs	Often the biggest win
Random oversampling	Duplicate minority rows	Risk of overfitting duplicates
SMOTE	Create synthetic minority points between neighbours	Numeric features; can create noise near class overlap
Undersampling	Drop majority rows	Fast; loses information
Anomaly detection	Model "normal" only	When positives are extremely rare or unlabelled

BALANCED CLASS WEIGHT

$$w_c = n_samples / (n_classes \times n_c)$$

With 10,000 rows and 500 positives, the positive weight is $10,000 / (2 \times 500) = 10$.

COMMON TRAP

Resampling (SMOTE or oversampling) **before** the train/test split, or outside the CV folds. Synthetic copies of test points leak into training. Resample only the training fold, for example with an imblearn Pipeline.

After resampling or reweighting, the predicted probabilities no longer reflect true rates. Recalibrate them if you need real probabilities.

8 Unsupervised Learning

8.1 Clustering algorithms

Algorithm	Finds	Needs k?	Strengths	Weaknesses
k-means	Spherical, similar-sized clusters	Yes	Fast; scales well	Needs scaling; sensitive to outliers and initialization; poor with odd shapes
Hierarchical (agglomerative)	A tree of nested clusters (dendrogram)	No (cut the tree)	Visual; any k from one run	Slow on large data
DBSCAN	Dense regions of any shape; marks noise	No (eps, min_samples)	Arbitrary shapes; finds outliers	Struggles with varying densities; sensitive to eps
Gaussian mixture	Elliptical clusters with soft membership	Yes	Probabilistic assignments	Assumes Gaussian shapes; slower

- **Choosing k:** the **elbow method** (where the drop in within-cluster sum of squares flattens) and the **silhouette score** (−1 to 1; higher means better-separated clusters).
- Always scale features for k-means, and use k-means++ initialization with several restarts (n_init).
- Validate clusters with the business: are the segments distinct, stable and actionable?

8.2 Dimensionality reduction

- **PCA** finds orthogonal directions of maximum variance. Standardize first, otherwise large-scale features dominate. Keep enough components to explain, say, 90–95% of the variance.
- PCA helps with visualization, noise reduction, multicollinearity and speed. The cost is interpretability, because components mix the original features.
- **PCA vs feature selection:** selection keeps original, interpretable features. PCA creates new combined ones.
- **t-SNE and UMAP:** non-linear 2-D or 3-D embeddings for *visualization only*. Distances between far-apart clusters are not meaningful, and they are not suitable as model features in production.

8.3 Anomaly detection

- **Isolation Forest:** anomalies are isolated in fewer random splits. It is fast on high-dimensional data. Set contamination to the expected anomaly rate.
- **Statistical rules:** z-score or IQR on single features. Simple but univariate.
- **Density or distance based:** Local Outlier Factor, DBSCAN noise points.
- **One-class models / autoencoders:** learn "normal" and flag large reconstruction errors.

9 Time Series, Text, Recommenders and Deep Learning

9.1 Time series

- **Components:** trend, seasonality (daily, weekly, yearly, Ramadan and Hajj seasons), cycles, noise.
- **Never split randomly.** Train on the past and validate on the future (time-series split / forward chaining).
- **Lag and rolling features:** value last week, 7-day rolling mean. Compute them only from data before the forecast time.
- **Baselines:** naive (last value), seasonal naive (same day last week). A model that cannot beat these is not useful.
- **Stationarity:** constant mean and variance over time. Differencing removes trend, as required by ARIMA-type models.

9.2 Text (NLP)

Approach	When
TF-IDF + linear model	Fast, strong baseline; small data; keyword-driven tasks such as spam or ticket routing
Word or sentence embeddings	Meaning and similarity matter (search, clustering)
Fine-tuned pretrained transformer	Best accuracy on nuanced language; needs more compute; Arabic-specific models help

9.3 Recommender systems

- **Collaborative filtering:** "users like you also liked". It needs interaction history, so it suffers from **cold start** for new users and items.
- **Content-based:** recommends items similar to what the user liked, using item attributes. It works for new items.
- **Hybrid:** combines both. For new users, a common fallback is popular or trending items.

9.4 Neural networks basics

- Deep learning pays off with large data and unstructured inputs (images, audio, text). On tabular data, gradient boosting is usually as good or better, and cheaper.
- **Transfer learning:** start from a pretrained model and fine-tune it on your data. It is essential when labelled data is limited.
- **Learning rate:** too high and the loss diverges or oscillates; too low and training is very slow.
- **Overfitting controls:** dropout, weight decay (L2), data augmentation, early stopping.
- **Batch size:** larger batches are faster per epoch but need more memory. For out-of-memory errors, reduce the batch size or use gradient accumulation.

10 Evaluating Classification Models

10.1 The confusion matrix

	Predicted positive	Predicted negative
Actual positive	TP true positive	FN false negative (missed)
Actual negative	FP false positive (false alarm)	TN true negative

Metric	Formula	Question it answers
Accuracy	$(TP + TN) / \text{total}$	How often is the model right overall?
Precision	$TP / (TP + FP)$	When it says positive, how often is it right?
Recall (sensitivity, TPR)	$TP / (TP + FN)$	Of all real positives, how many did it catch?
Specificity (TNR)	$TN / (TN + FP)$	Of all real negatives, how many did it clear?
False positive rate	$FP / (FP + TN) = 1 - \text{specificity}$	How many negatives were falsely flagged?
F1-score	$2 \cdot P \cdot R / (P + R)$	A balance of precision and recall (harmonic mean)
F-beta	$(1 + \beta^2) \cdot P \cdot R / (\beta^2 \cdot P + R)$	$\beta > 1$ weights recall more (F2); $\beta < 1$ weights precision more (F0.5)

WORKED EXAMPLE

TP = 80, FN = 20, FP = 40, TN = 860 (total 1,000).

Precision = $80/120 = 66.7\%$ · Recall = $80/100 = 80\%$ · Specificity = $860/900 = 95.6\%$
· Accuracy = $940/1000 = 94\%$ · F1 = 72.7%

10.2 Precision or recall? Decide by the cost of errors

Use case	Costlier error	Prioritize
Cancer or disease screening	Missing a sick patient (FN)	Recall
Fraud detection (with review capacity)	Missing fraud (FN)	Recall, at acceptable precision
Spam filter	Losing a real email (FP)	Precision
Retention campaign with a limited budget	Wasting calls on non-churners (FP)	Precision in the top-k contacted
Automatic account blocking	Blocking a good customer (FP)	Precision
Predictive maintenance of critical equipment	A missed failure (FN)	Recall

ACCURACY PARADOX

With 1% fraud, a model that predicts "not fraud" for everything scores 99% accuracy and 0% recall. Never judge an imbalanced problem by accuracy.

10.3 Thresholds, ROC and PR curves

- Most classifiers output a probability. The default threshold of 0.5 is arbitrary. **Lowering the threshold** raises recall and lowers precision; **raising it** does the opposite.
- **ROC curve:** TPR vs FPR across all thresholds. **ROC-AUC** is the probability that a random positive is ranked above a random negative (0.5 = random, 1.0 = perfect).
- **PR curve:** precision vs recall. **PR-AUC / average precision** is more informative than ROC-AUC when positives are rare, because ROC-AUC can look excellent while precision is poor.
- AUC measures *ranking* quality and ignores the threshold and calibration.

CHOOSING A THRESHOLD BY COST

$$\text{Total cost} = \text{FP} \times \text{cost}_{\text{FP}} + \text{FN} \times \text{cost}_{\text{FN}}$$

Compute it on validation data at each candidate threshold and pick the minimum. When probabilities are calibrated, the break-even threshold is $\text{cost}_{\text{FP}} / (\text{cost}_{\text{FP}} + \text{cost}_{\text{FN}})$.

10.4 Calibration

- A model is **calibrated** if, among cases predicted at 70%, about 70% are actually positive. You can check this with a reliability diagram.
- Calibration matters when probabilities drive decisions: expected-loss pricing, risk scores, cost-based thresholds.
- **Methods:** Platt scaling (sigmoid) for small data, isotonic regression for larger data. Fit them on held-out data.
- **Metrics:** Brier score = $\text{mean}((p - y)^2)$, where lower is better, and log loss, which heavily punishes confident wrong predictions.
- Boosted trees, SVMs, Naive Bayes, and models trained on resampled data are often poorly calibrated.

10.5 Multi-class metrics

Averaging	How	Use when
Macro	Compute the metric per class, then take the simple average	Every class matters equally, including rare ones
Weighted	Per-class average weighted by class support	Reflect the overall class mix
Micro	Pool all TP, FP and FN across classes	Overall performance; equals accuracy for single-label problems

10.6 Ranking and campaign metrics

- **Precision@k:** the share of true positives among the top k scored cases. Use it when you can act on only k cases.
- **Lift** = response rate in the targeted group $\div$ overall response rate. A top-decile lift of 3 means three times better than random targeting.
- **Cumulative gains:** the share of all positives captured when contacting the top x% of cases.

11 Regression Metrics and Validation Design

11.1 Regression metrics

Metric	Formula	Properties
MAE	$\text{mean } y - \hat{y} $	In target units; every error counts in proportion; less sensitive to outliers
MSE	$\text{mean } (y - \hat{y})^2$	Squared units; punishes large errors heavily
RMSE	$\sqrt{\text{MSE}}$	In target units; still punishes large errors more than MAE
MAPE	$\text{mean } y - \hat{y} / y $	Percentage, easy to explain; undefined at $y = 0$ and explodes near 0; penalizes over-forecasts more
R^2	$1 - SS_{\text{res}} / SS_{\text{tot}}$	Share of variance explained; can be negative when the model is worse than predicting the mean
Adjusted R^2	$1 - (1 - R^2)(n - 1)/(n - p - 1)$	Penalizes adding useless predictors
Median absolute error	$\text{median } y - \hat{y} $	Very robust to corrupt records

CHOOSING A REGRESSION METRIC

If large errors are disproportionately costly, use RMSE. If every unit of error costs the same, use MAE. If stakeholders need percentages and targets are never near zero, use MAPE (or WAPE).

11.2 Train, validation and test sets

- **Training set:** fit the model's parameters.
- **Validation set (or cross-validation):** choose models, features, hyperparameters and thresholds.
- **Test set:** one final, unbiased estimate. Touch it once. Repeatedly checking the test set and adjusting turns it into a validation set and makes the estimate optimistic.

11.3 Cross-validation strategies

Strategy	Use when
Hold-out split	Very large data; quick experiments
K-fold ($k = 5$ or 10)	General case; more stable estimate than a single split
Stratified K-fold	Classification, especially imbalanced; keeps the class ratio in every fold
Group K-fold	Several rows per customer, patient or device; the same group must never be in both train and validation
Time-series split (forward chaining)	Temporal data; always train on the past, validate on the future

Strategy	Use when
Leave-one-out	Very small datasets; high variance and costly
Nested CV	Tuning hyperparameters <i>and</i> reporting an unbiased performance estimate

11.4 Data leakage

Leakage happens when information that would not be available at prediction time reaches training. The results look excellent offline and collapse in production.

Type	Example	Prevention
Target leakage	"refund_issued" used to predict fraud; "discharge_date" used to predict length of stay	Ask of each feature: is it known at the moment of prediction?
Preprocessing leakage	Scaling, imputing or selecting features on all data before splitting	Fit transformers inside a Pipeline on training folds only
Temporal leakage	Random split on time-ordered data; features using future values	Time-based split; point-in-time feature computation
Group leakage	The same patient's scans in both train and test	GroupKFold / split by entity
Duplicate leakage	Near-identical rows on both sides of the split	Deduplicate before splitting
Resampling leakage	SMOTE before splitting	Resample the training fold only

"TOO GOOD TO BE TRUE"

An unexpectedly high score (e.g. AUC 0.99 on a hard problem) or one feature with overwhelming importance is a leakage warning sign. Investigate before celebrating.

11.5 Comparing models fairly

- Use the same folds and data for every candidate, and always compare against a simple baseline.
- Look at the variability across folds. A 0.2% difference inside the fold-to-fold noise is not a real improvement.
- Do **error analysis** by slicing performance by segment (region, device, customer type). A good average can hide a failing group.
- Check that the test data represents production (same time period, population and data sources).
- An offline metric is a proxy. Confirm business value online (A/B test, shadow deployment).

12 Production ML (MLOps)

12.1 Serving patterns

	Batch inference	Online (real-time) inference
How	Score many records on a schedule and store the results	Score each request instantly through an API
Latency	Minutes to hours	Milliseconds
Examples	Nightly churn scores, weekly demand forecasts	Card fraud at checkout, search ranking
Complexity	Lower	Higher: latency, scaling, availability

12.2 Packaging and reproducibility

- **Serialization:** pickle/joblib are Python-specific, can break across library versions, and must **never be loaded from untrusted sources** because loading can execute code. ONNX gives a portable, framework-independent format.
- **REST API:** wrap the model with FastAPI or Flask, validate the input schema, and return predictions with a version tag.
- **Docker:** an *image* is the packaged environment (code + dependencies); a *container* is a running instance of it. It gives the same behaviour on every machine.
- **Reproducibility:** pin dependency versions, fix random seeds, version the data and models, and track experiments (parameters, metrics, artifacts) with a tool such as MLflow.
- **Model registry:** the central store of model versions with stages (staging, production, archived) and a lineage back to data and code.
- **CI/CD for ML:** automated tests for code, data validation (schema, ranges, nulls), and model quality gates before promotion.

12.3 Training-serving skew and feature stores

Training-serving skew happens when features are computed differently in training (e.g. a Python batch job) and serving (e.g. a Java service), so the model receives inputs with a different meaning. Prevent it by sharing one feature definition, for example a **feature store** that serves the same features offline for training and online for inference. Log the online features and compare them with the offline values.

12.4 Deployment strategies

Strategy	How	Why
Shadow	The new model scores real traffic, but its outputs are not used	Validate on live data with zero user risk
Canary	Send a small share (e.g. 5%) of traffic to the new model, then increase	Limit the blast radius; roll back quickly

Strategy	How	Why
Blue-green	Two full environments; switch traffic at once	Instant rollback
A/B test	Randomly split users and compare business metrics	Measure the real business impact

12.5 Monitoring and drift

Type	What changes	Example	Typical response
Data (covariate) drift	$P(X)$: the input distribution	A new customer segment joins after a campaign	Investigate; retrain if performance drops
Concept drift	$P(y X)$: the relationship between inputs and target	Fraudsters change tactics; the same features mean something different	Retrain on recent data; revisit features
Label (prior) shift	$P(y)$: the base rate	Flu prevalence rises in winter while symptoms per case stay the same	Recalibrate or adjust thresholds

- **Detecting drift:** PSI (population stability index; < 0.1 stable, $0.1-0.25$ moderate, > 0.25 significant), the Kolmogorov–Smirnov test for numeric features, and chi-square for categorical ones.
- **Delayed labels:** when true outcomes arrive weeks later, monitor input drift and prediction distributions as early warnings.
- **Operational metrics:** latency, error rate, throughput, cost.
- **Retraining:** scheduled (e.g. monthly) or triggered (by drift or a performance drop). Choose the cheapest policy that keeps performance above the requirement.
- **Alerting:** set thresholds that are actionable. Too many alerts cause alert fatigue.
- **Feedback loops:** model decisions can shape future training data (e.g. only approved loans get repayment labels). Keep a small random exploration or control group.

12.6 Scale and cost

- **Smaller, faster models:** distillation (a small student learns from a large teacher), quantization (lower-precision weights), pruning.
- **Caching** repeated predictions, and batching requests.
- Prefer the simplest model that meets the requirement. A 0.3% accuracy gain rarely justifies triple the serving cost.

13 Responsible AI

13.1 Explainability

- **Global explanations:** how the model behaves overall, for example permutation importance or SHAP summary plots.
- **Local explanations:** why *this* prediction was made, for example SHAP values or LIME for a single customer. They are needed to explain individual decisions such as a rejected loan.
- **Caveats:** importance is not causation, and correlated features share or split importance, so permute them as a group.
- For high-stakes regulated decisions, an inherently interpretable model is often preferable to explaining a black box after the fact.

13.2 Fairness and bias

- Bias can enter through historical data, unrepresentative samples, proxy features (a postcode acting as a proxy for a protected attribute), or label definitions.
- Removing the sensitive attribute is not enough, because proxies remain. Measure performance and error rates **per group**.
- Common checks: demographic parity (similar positive rates) and equal opportunity (similar recall across groups). These criteria can conflict, so choose them with stakeholders.

13.3 Privacy and data protection

- **Data minimization:** collect and keep only what the purpose requires.
- **Consent and purpose limitation:** use personal data only for the purposes people agreed to.
- **Pseudonymization** replaces identifiers with codes, but the data is still personal because it is re-linkable. **Anonymization** removes the ability to identify anyone. Beware re-identification through combinations of quasi-identifiers such as age, postcode and gender.
- **Access control and encryption;** never put secrets or personal data in logs or notebooks shared widely.
- Follow the applicable regulation. In Saudi Arabia this is the Personal Data Protection Law (PDPL).

13.4 Human oversight and documentation

- **Human-in-the-loop** for high-stakes decisions: the model prioritizes or recommends, and a person decides, especially for low-confidence cases.
- **Model cards** document intended use, training data, performance by group, limitations and ethical considerations.

14 Quick Decision Sheets

One-glance answers to the decisions that come up most often in scenario questions.

14.1 Which metric?

If the scenario says...	Think
Missing a positive is very costly (disease, fraud, failures)	Recall (or F2)
False alarms are very costly (blocking customers, spam)	Precision (or F0.5)
Both errors matter; classes are imbalanced	F1, PR-AUC
Only the top k cases can be acted on	Precision@k, lift
Ranking quality regardless of threshold	ROC-AUC (PR-AUC if positives are rare)
Probabilities themselves are used	Log loss, Brier score, calibration curve
Explicit costs per FP and FN are given	Expected cost; choose the threshold that minimizes it
Regression where large errors hurt most	RMSE
Regression, errors cost linearly, outliers present	MAE (median absolute error if records are corrupt)
Management wants percentage error, no zero targets	MAPE / WAPE

14.2 Which model?

If the scenario says...	Think
Decisions must be explained to regulators or customers	Logistic regression, shallow tree, scorecard
Tabular data, maximum accuracy	Gradient boosting
Strong baseline quickly, little tuning	Random forest or logistic regression
Short text classification, small labelled set	TF-IDF + linear model; or a pretrained transformer
Images with few labels	Transfer learning from a pretrained CNN or vision transformer
No labels, find groups	k-means (spherical), DBSCAN (odd shapes, noise)
Very rare or unlabelled anomalies	Isolation Forest, anomaly detection
New users with no history	Content-based or popularity fallback (cold start)

14.3 Which validation?

Data looks like...	Use
Independent rows, imbalanced classes	Stratified K-fold

Data looks like...	Use
Several rows per customer, patient or device	Group K-fold
Ordered in time	Time-series split (train past, validate future)
Tuning and reporting on a small dataset	Nested cross-validation

14.4 Which preprocessing?

Situation	Choice
Nominal category, few levels, linear model	One-hot encoding
Thousands of categories	Target encoding (out-of-fold), frequency, or hashing
Ordered categories	Ordinal encoding
Features with outliers	RobustScaler or log transform
Tree-based model	No scaling needed
Missing values, skewed numeric feature	Median imputation (+ missing indicator)
Missingness depends on the value itself (MNAR)	Missing indicator + domain investigation

14.5 Diagnosing a model

Symptom	Likely cause	Action
Train high, validation much lower	Overfitting	Regularize, simplify, more data, early stopping
Train and validation both low	Underfitting	Richer model or features
Validation excellent, production poor	Leakage or training-serving skew	Audit features and pipelines
Performance decays month after month	Drift	Monitor, retrain, recalibrate
High accuracy, low recall on rare class	Class imbalance	Class weights, threshold, PR metrics
AUC stable, predicted rate far from actual	Label shift or poor calibration	Recalibrate

15 Exam Strategy

- **Pace:** about 90 seconds per question (140 questions in 3.5 hours). If a question passes two minutes, flag it and move on.
- **Read the last line first** to see what is being asked (the BEST action, what to do FIRST, the MOST likely cause), then scan the scenario for the deciding constraint.
- **Look for the constraint that decides the answer:** interpretability, latency, cost of a false negative vs a false positive, class imbalance, time order, repeated entities, small data, privacy.
- **Long scenarios contain irrelevant details** (team names, dashboards, maintenance windows). Ignore them.
- **Eliminate first:** answers that leak data, touch the test set, use accuracy on imbalanced data, or skip a baseline are usually wrong.
- **"Select TWO"** questions: judge each option independently as true or false.
- **Never leave an answer blank.** Flag, guess, and return if time allows.

16 Glossary

Term	Arabic	Meaning
Accuracy	الدقة الكلية	Share of all predictions that are correct
A/B test	اختبار أ/ب	Randomized experiment comparing two variants
Bagging	التجميع بإعادة المعاينة	Training models on bootstrap samples and averaging them
Baseline	خط الأساس	Simple reference model or rule to beat
Bias (model)	الانحياز	Error from overly simple assumptions
Boosting	التعزيز	Sequential models that correct previous errors
Calibration	المعايرة	Agreement between predicted probabilities and observed rates
Class imbalance	عدم توازن الفئات	One class far rarer than the others
Clustering	التجميع العنقودي	Grouping similar records without labels
Concept drift	انجراف المفهوم	Change in the relationship between inputs and target
Confidence interval	فترة الثقة	Range that quantifies uncertainty of an estimate
Confounder	متغير مُربك	Variable that influences both cause and effect
Confusion matrix	مصفوفة الارتباك	Table of TP, FP, FN, TN
Cross-validation	التحقق المتقاطع	Repeated train/validate splits for a stable estimate
Data drift	انجراف البيانات	Change in the input distribution over time
Data leakage	تسرب البيانات	Information unavailable at prediction time reaching training
Dimensionality reduction	تقليل الأبعاد	Representing data with fewer features
Early stopping	الإيقاف المبكر	Stop training when validation stops improving
Encoding	الترميز	Converting categories into numbers
Ensemble	النماذج المجمعّة	Combining several models
Feature	الخاصية / المتغير المستقل	Input variable used by a model
Feature store	مستودع الخصائص	Shared system serving the same features for training and inference
Hyperparameter	المعامل الفائق	Setting chosen before training (depth, learning rate)
Imputation	تعويض القيم المفقودة	Filling missing values
Inference	الاستدلال / التنبؤ	Using a trained model to make predictions
Interpretability	قابلية التفسير	How easily humans understand model decisions
Label	الوسم / المتغير الهدف	The outcome the model learns to predict
Learning rate	معدل التعلّم	Step size of each training update

Term	Arabic	Meaning
Outlier	القيمة الشاذة	Observation far from the rest
Overfitting	الإفراط في التوفيق	Memorizing training data; poor generalization
p-value	القيمة الاحتمالية	Probability of data this extreme if H_0 is true
Pipeline	خط المعالجة	Chained preprocessing and model fitted together
Precision	الضبط	$TP / (TP + FP)$
Recall	الاستدعاء / الحساسية	$TP / (TP + FN)$
Regularization	التنظيم	Penalty that discourages complex models
Residual	البواقي	Actual minus predicted value
Sampling bias	تحيز العينة	Sample not representative of the population
Scaling	توحيد المقياس	Putting features on comparable ranges
Serialization	حفظ النموذج	Saving a trained model to a file
Specificity	النوعية	$TN / (TN + FP)$
Stratification	التقسيم الطبقي	Preserving group proportions when splitting or sampling
Threshold	عتبة القرار	Probability cut-off for a positive prediction
Training-serving skew	اختلاف التدريب عن التشغيل	Features computed differently in training and production
Underfitting	ضعف التوفيق	Model too simple to capture the pattern
Variance (model)	التباين	Sensitivity to the particular training sample